```
module async_fifo #(
    parameter DEPTH = 64,  //0--63  6bit[5:0]
    parameter WIDTH = 16
)(
    //wr
    input                i_wclk,
    input                i_wrstn,
    input                i_wren,
    input  [WIDTH-1:0]      i_wdata,
    //rd
    input                i_rclk,
    input                i_rrstn,
    input                i_rden,
    output [WIDTH-1:0]      o_rdata,
    output                  o_full,
    output                  o_empty
);

    reg  [$clog2(DEPTH-1):0]   r_wrptr;     //[6:0]
    reg  [$clog2(DEPTH-1):0]   r_rdptr;     //[6:0]
    wire [$clog2(DEPTH-1)-1:0] w_wraddr;    //[5:0]
    wire [$clog2(DEPTH-1)-1:0] w_rdaddr;    //[5:0]

    assign w_wraddr = r_wrptr[$clog2(DEPTH-1)-1:0];
    assign w_rdaddr = r_rdptr[$clog2(DEPTH-1)-1:0];

    always @(posedge i_wclk or negedge i_wrstn) begin
        if (!i_wrstn)
            r_wrptr <= 'd0;
        else if (i_wren && !o_full)
            r_wrptr <= r_wrptr + 1'b1;
        else
            r_wrptr <= r_wrptr;
    end

    always @(posedge i_rclk or negedge i_rrstn) begin
        if (!i_rrstn)
            r_rdptr <= 'd0;
        else if (i_rden && !o_empty)
            r_rdptr <= r_rdptr + 1'b1;
        else
            r_rdptr <= r_rdptr;
    end

    //使用时序逻辑转化成格雷码，降低组合逻辑延迟
    //有利于综合之后时钟跑的更快
    wire [$clog2(DEPTH-1):0] w_wrptr_gray;
    wire [$clog2(DEPTH-1):0] w_rdptr_gray;

    assign w_wrptr_gray = r_wrptr ^ (r_wrptr>>1);
    assign w_rdptr_gray = r_rdptr ^ (r_rdptr>>1);

    //单bit跨时钟域CDC
    reg [$clog2(DEPTH-1):0] r_rdptr_gray_d0;
    reg [$clog2(DEPTH-1):0] r_rdptr_gray_d1;

    always @(posedge i_wclk or negedge i_wrstn) begin
        if (!i_wrstn) begin
            r_rdptr_gray_d0 <= 'd0;
            r_rdptr_gray_d1 <= 'd0;
        end
        else begin
            r_rdptr_gray_d0 <= w_rdptr_gray;
            r_rdptr_gray_d1 <= r_rdptr_gray_d0;
        end
    end

    reg [$clog2(DEPTH-1):0] r_wrptr_gray_d0;
    reg [$clog2(DEPTH-1):0] r_wrptr_gray_d1;

    always @(posedge i_rclk or negedge i_rrstn) begin
        if (!i_rrstn) begin
            r_wrptr_gray_d0 <= 'd0;
            r_wrptr_gray_d1 <= 'd0;
        end
        else begin
            r_wrptr_gray_d0 <= w_wrptr_gray;
            r_wrptr_gray_d1 <= r_wrptr_gray_d0;
        end
    end

    //在rd_clk下产生的信号
```

```
    assign o_empty = (w_rdptr_gray == r_wrptr_gray_d1);

    //在wr_clk下产生的信号
    assign o_full = (w_wrptr_gray == {~r_rdptr_gray_d1[$clog2(DEPTH-1):$clog2(DEPTH-1)-1],r_rdptr_gray_d1[$clog2(DEPTH-1)-2:0]});

    ram #(
        .DATA_WIDTH(WIDTH),
        .DATA_DEPTH(DEPTH)
    ) u_ram (
        .i_wrclk(i_wclk),
        .i_wrstn(i_wrstn),
        .i_wren(~o_full && i_wren),
        .i_waddr(w_wraddr),
        .i_wdata(i_wdata),
        .i_rdclk(i_rclk),
        .i_rdrstn(i_rrstn),
        .i_rden(~o_empty && i_rden),
        .i_raddr(w_rdaddr),
        .o_rdata(o_rdata)
    );

endmodule

module ram #(
    parameter DATA_WIDTH = 8,
    parameter DATA_DEPTH = 64
)(
    //wr
    input                           i_wrclk,
    input                           i_wrstn,
    input                           i_wren,
    input     [$clog2(DATA_DEPTH-1)-1:0]   i_waddr,
    input     [DATA_WIDTH-1:0]          i_wdata,
    //rd
    input                           i_rdclk,
    input                           i_rdrstn,
    input                           i_rden,
    input     [$clog2(DATA_DEPTH-1)-1:0]   i_raddr,
    output    [DATA_WIDTH-1:0]          o_rdata
);

    reg  [DATA_WIDTH-1:0] dual_ram[0:DATA_DEPTH-1];
    reg  [DATA_WIDTH-1:0] r_rdata;
    wire [DATA_WIDTH-1:0] test_dual_ram0;
    wire [DATA_WIDTH-1:0] test_dual_ram127;

    assign test_dual_ram0   = dual_ram[0];
    assign test_dual_ram127 = dual_ram[127];

    integer i;
    always @(posedge i_wrclk or negedge i_wrstn) begin
        if (!i_wrstn)
            for (i = 0; i < DATA_DEPTH; i = i + 1)
                dual_ram[i] = 'd0;
        else if (i_wren)
            dual_ram[i_waddr] <= i_wdata;
        else
            dual_ram[i_waddr] <= dual_ram[i_waddr];
    end

    always @(posedge i_rdclk or negedge i_rdrstn) begin
        if (!i_rdrstn)
            r_rdata <= 'd0;
        else if (i_rden)
            r_rdata <= dual_ram[i_raddr];
        else
            r_rdata <= r_rdata;
    end

    assign o_rdata = r_rdata;

endmodule
```